COMPANION NOTE · PREPARED FOR THE PLATFORM TEAM · DATA ENGINEER

Different product, **same plumbing.**

Why Lifepal's regulated insurance marketplace maps onto Abound's consumer lending platform.

Lifepal sold regulated insurance; Abound lends, using Open Banking data and AI to decide. Both run on a transactional database that the business, its reports and its risk models all depend on, and that nobody should query for analytics. In both, data changes after the fact, a customer can ask to be forgotten, and a real financial decision depends on the number being right. The problems that follow are the ones NEXUS was built to solve.



ONE FLOW, TWO BUSINESSES

Abound side from public material

01 Source
Lifepal · PostgreSQL and MySQL
Abound · Aurora PostgreSQL / MySQL

→

02 Capture
Lifepal · DMS and Debezium
Abound · AWS DMS

→

03 Apply
Lifepal · Spark and Glue on Delta
Abound · Spark on Glue, Iceberg

→

04 Prove
Lifepal · 500+ checks in CI
Abound · reconciliation and rules

→

05 Decide
Lifepal · risk and fraud models
Abound · underwriting and calibration

THE SAME PROBLEM

how I handled it · where it shows here

THE PROBLEM	AT LIFEPAL	HERE
Analytics on the production DB	Moved reporting, ad-hoc analytics and model training onto the lake	The role's first 6–12 months; gold tables in gold.py
Changes arrive twice, late, out of order	Merged on the source log position, idempotent under replay	change_seq ordering; a full replay changed 0 rows
Rows that never arrive	Reconciliation against a read replica, then a reload	16 rows caught, reloaded, 18/18 checks pass
Features as they were known	Point-in-time training sets for risk and fraud	PIT income differs for 72 of 111 decisions
Cost without losing reliability	$11k a month against a $19k projection	90% less scanned per customer read
Right to erasure	PII masking, role-based access, deletes through CDC	tombstones, bronze purge, expire_snapshots

STATED PLAINLY

what's real

Real: the Lifepal outcomes, from my CV and production work, and all the code in the repository, which runs end to end in CI.

Synthetic: every customer, loan, repayment and transaction. The Abound side comes from the job description and public information; no Abound systems or data were used.

Depth: DMS, Spark, Glue, Aurora/RDS PostgreSQL and MySQL, Lambda and ECS are production experience. Delta is my production table format; this repository is my Iceberg work. Athena is working level.

QUESTIONS I'D BRING

for the Platform team

- Which workloads still read Aurora directly, and which would Data Science move first?
- How are DMS gaps and restarts detected today: heartbeat, volume, or reconciliation?
- Where do model calibration and its features live now, and how fresh do they need to be?
- Which AWS line items are growing fastest: Glue, Athena, S3 or DMS?
- How do erasure requests reach the lake today?

Described at the level I can share publicly; I'm happy to walk through details in conversation. rfim.github.io/nexus-abound